

Key-Value Store v6

Sharding, router e hotspot

Architetture dei Sistemi Distribuiti

Lezione 15

Fin qui ci siamo chiesti:

- chi scrive?
- quante repliche devono confermare?

Ora la domanda cambia natura:

- dove vive la chiave?

Stiamo passando da replica dell'intero dataset a partizionamento del key space.

Il router traduce:

- API uniforme lato client;
- topologia partizionata lato backend.

Gli shard diventano proprietari di sottoinsiemi di chiavi. Quindi la stessa richiesta client puo' seguire percorsi interni diversi in base alla chiave.

Regola astratta:

$$shard = hash(key) \bmod numero_di_shard$$

Conseguenze:

- stessa chiave, stesso shard;
- chiavi diverse, partizioni potenzialmente diverse;
- uniformita' dell'API ma non uniformita' del percorso interno.

Restano locali a uno shard:

- GET
- SET
- DELETE
- EXISTS
- INCR

Questo e' il vantaggio principale dello sharding: molte operazioni non devono piu' coinvolgere l'intero cluster.

Diventano globali:

- KEYS
- STATS

Il router deve:

- interrogare tutte le partizioni;
- raccogliere le risposte;
- comporre un risultato unico.

Il partizionamento cambia quindi anche il costo osservabile delle operazioni.

Perche' WHERE E' Importante

Il comando `WHERE <key>` rende osservabile:

- lo shard scelto dal router;
- il fatto che il routing e' parte del contratto;
- che la stessa API non implica piu' lo stesso backend per tutte le chiavi.

`WHERE` e' didatticamente prezioso perche' mostra la topologia sotto l'interfaccia.

Operazioni Uniformi, Costi Non Uniformi

Dal lato client:

- GET alpha
- GET beta

sembrano simmetriche.

Dal lato interno possono essere diverse:

- shard diversi;
- code diverse;
- hotspot diversi;
- latenze diverse.

Anche con molti shard, una chiave molto calda resta su una singola partizione. Quindi:

- il cluster puo' avere molti nodi;
- ma il collo di bottiglia puo' restare locale a uno shard;
- il throughput globale non elimina automaticamente la concentrazione di carico.

Perche' KEYS Cambia Significato

Nel nodo singolo KEYS era locale. Con lo sharding diventa scatter-gather:

- costo proporzionale al numero di shard;
- dipendenza dalla disponibilita' di tutte le partizioni;
- maggiore ambiguita' semantica in caso di errori parziali.

La stessa operazione sintattica ha quindi cambiato natura architetturale.

Possiamo vedere il router come:

- Receive -> ComputeTarget -> Forward -> CollectReply -> Return

Per le operazioni globali:

- Receive -> Broadcast -> CollectMany -> Aggregate -> Return

Gia' qui si vede che non tutte le richieste hanno piu' la stessa forma interna.

Routing:

- ① usare WHERE alpha e WHERE beta;
- ② scrivere le due chiavi;
- ③ osservare STATS;
- ④ verificare che la topologia diventi parte del ragionamento sul comportamento.

Hotspot:

- 1 ripetere molti INCR sulla stessa chiave;
- 2 osservare quale shard concentra il carico;
- 3 discutere perche' il partizionamento non elimina automaticamente la concentrazione.

Costo di KEYS:

- ① distribuire chiavi su piu' shard;
- ② eseguire KEYS;
- ③ osservare che il router deve interrogare tutte le partizioni;
- ④ discutere perche' questa operazione non puo' piu' essere considerata "banale".

Aggiungere un nuovo shard implica:

- cambiare la funzione di routing;
- spostare chiavi già esistenti;
- evitare inconsistenze tra destinazione teorica e posizione reale.

La lezione 15 introduce il partizionamento statico. La lezione 16 mostrerà il costo semantico del partizionamento dinamico.

Con lo sharding la stessa API resta uniforme, ma:

- il percorso cambia in base alla chiave;
- il costo non e' piu' uniforme;
- alcune operazioni sono locali e altre globali;
- un cluster piu' grande puo' ancora soffrire di squilibri forti.

Lo spazio delle chiavi diventa parte del contratto del sistema.